bitop/v1 on a 16–40-bit prime ladder: index calculus with a per-size base

`research/bitop_ladder_20261019` · protocol frozen 2026-10-19 · report 2026-10-20

Question

The bitop/v1 study (PR #1970) fitted exponents with intervals of about ±0.1 from 88 runs an arm on 16–26-bit curves, and ran index calculus at fixed bases only. This study asks:

1. **Precision.** What are the exponents with about 2,000 runs an arm on 25 curves from 16 to 40 bits?
2. **The fair test.** When index calculus chooses its base per curve size on a separate train panel, does its exponent bend toward ½?

Everything here was preregistered in `PROTOCOL.md`, including the verdict rules.

Interpretation. `bitop/v1` is a reproducible accounting convention with frozen model weights, not a physical gate count, synthesized-circuit cost or circuit lower bound. "Completely priced" means `ecbench` reports no declared unpriced work in the captured counters; it does not prove that every executed operation was instrumented. Replay and the control checks validate specific properties of the records and comparisons, not full coverage or the physical validity of the primitive prices. Results below are comparisons under the stated convention and captured counters, with that limitation.

Method

- **Curves.** One `a = −3` prime-order curve per size from 16 to 40 bits (`prime_fast::find_a3_curve`); the 20/24/28/32/36/40-bit ones are the 2026-10-15 campaign's.
- **Phase A, train panel.** Target seed 20261019100: 8 targets per curve up to 34 bits, 4 from 35 to 40 bits.
  - Candidate bases come from a geometric grid with step $2^{\frac{1}{2}}$.
  - The winner is the base with the smallest mean bitop/v1 cost among bases whose train runs all verified and have no declared unpriced work in the captured counters. This is not proof of complete instrumentation.
  - A winner at a grid edge triggers a one-step extension, at most three per size and arm.
  - It took 46 sessions and 2,296 runs: 2,295 verified, and one crashed (the kernel's out-of-memory killer, most likely, in a far-from-optimal candidate).
- **Phase B, hold-out.** Target seed 20261019200, 80 targets a curve, 12,080 runs. Every run verified; `ecbench` reports no declared unpriced work in the captured counters. This does not prove complete instrumentation. Every session passes `ecbench verify --replay 2`.
- **Arms.**
  - rho (negation): the reference;
  - BSGS (negation) and kangaroo: the positive control;
  - an A/A rho;
  - the MITM pair table with the per-size base (16–40 bits);
  - the pair table at the fixed base 32 (16–28 bits);
  - root-finding with $m = 3$ and a per-size base (16–28 bits).
- **Measures.** `ecbench bound fit --unit bitop/v1` (two-stage cluster bootstrap, 2,000 resamples), `ecbench work`, `compare`, `table`, `band` and `amortize`. Counts only, at isolation L0, with one binary (`372ed2eb…`).

Selection (Phase A)

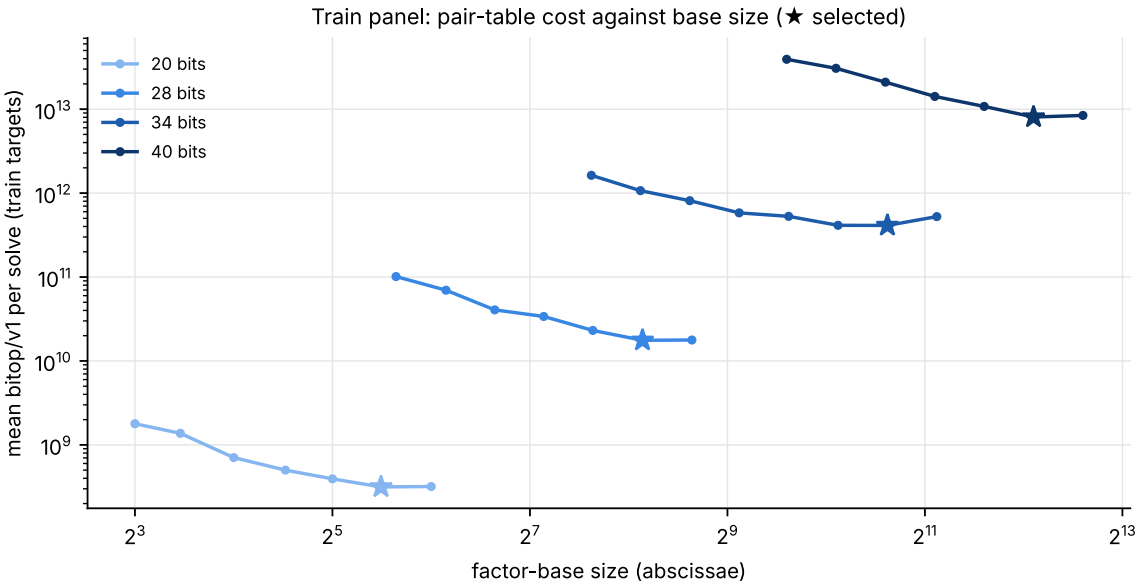


Figure 1: Train panel: mean bitop/v1 per solve against base size, pair table, four sizes (★ selected). Sessions `sessions/train-P*`; values from `select/selection.json`.

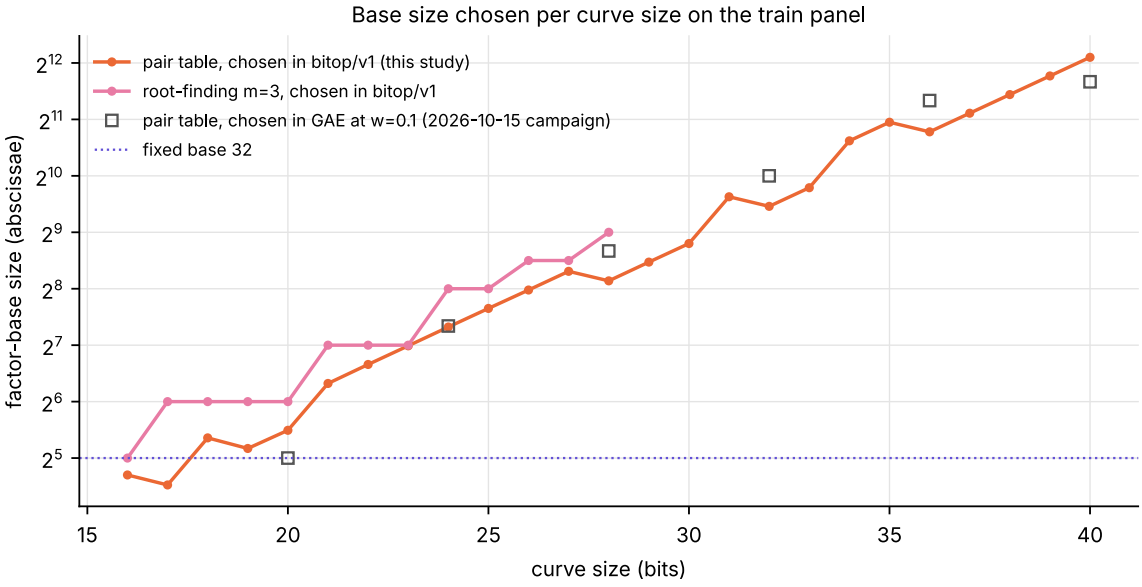


Figure 2: Selected base per size (`params.json`), with the campaign's GAE selection for comparison. The two agree within about a factor of 1.5 in both directions.

Exponents (Phase B)

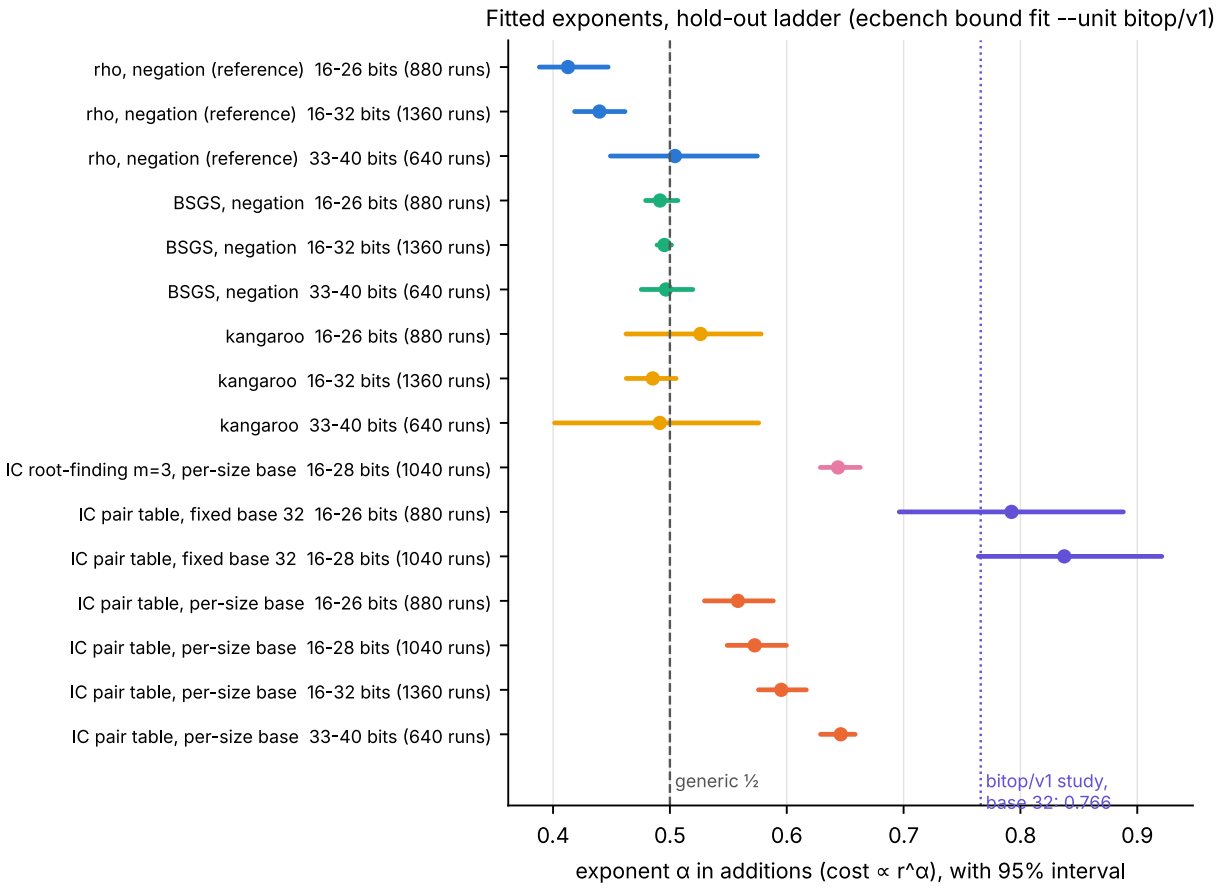


Figure 3: Exponents in additions with 95% intervals, from `analysis/bounds/*.json`. ecbench refuses one fit across its toy (≤ 32 bits) and medium tiers, so the fits are per tier (Deviation 3).

arm	16–26 bits	16–32 bits	33–40 bits
rho, negation	0.413 [0.388, 0.447]	0.440 [0.418, 0.462]	0.504 [0.449, 0.575]
BSGS, negation	0.492 [0.479, 0.507]	0.495 [0.489, 0.501]	0.497 [0.475, 0.520]
kangaroo	0.526 [0.462, 0.578]	0.485 [0.463, 0.505]	0.491 [0.401, 0.576]
IC pair, per-size base	0.558 [0.530, 0.588]	0.595 [0.576, 0.617]	0.646 [0.629, 0.659]
IC pair, base 32	0.792 [0.696, 0.888]	16–28: 0.838 [0.764, 0.921]	—
IC root-finding, per-size	—	16–28: 0.644 [0.629, 0.663]	—

Precision. Interval widths at 16–32 bits:

- BSGS: 0.012;
- the IC pair table: 0.041;
- rho: 0.043.

PR #1970's 16–26-bit ladder had widths of 0.07–0.31.

Positive control. It trips once. Rho's 16–26-bit interval excludes ½ by 0.053, above the protocol's 0.05 tolerance.

- **Other ranges:** rho at 16–32 bits misses ½ by 0.038, within the tolerance, and rho at 33–40 bits contains ½.
- **BSGS:** contains ½ in every range.
- **Pattern:** rho's constant falls with size (1.47, 1.33 and 1.05 times the floor). That is consistent with fixed per-walk overhead on small curves. This explanation was not tested.

A/A control. Uninformative: it reuses rho's seeds, so its runs are identical copies (Deviation 4).

Distance to rho

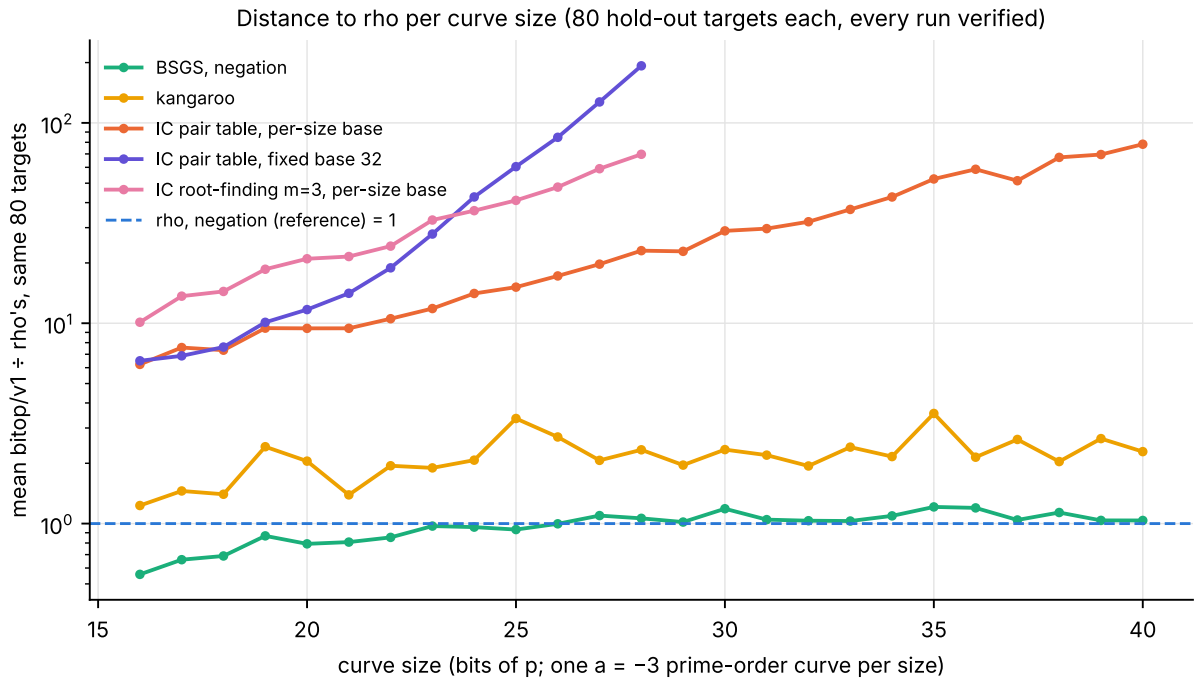


Figure 4: Mean bitop/v1 over rho's, per curve, 80 hold-out targets each (`analysis/gap_to_rho.json` from `ecbench work --counted`).

Verdicts (preregistered)

- **Step 2: bends toward ½.**
  - The per-size pair table's 16–26-bit upper bound, 0.588, is below the bitop/v1 study's fixed-base 0.766.
  - Its 16–28-bit interval, [0.549, 0.600], lies below the fixed base's from this study, [0.764, 0.921].
- **Success criterion: met.** This is a verified, measured improvement over the fixed-base index-calculus baseline under the declared bitop/v1 convention and recorded counters. The fixed base costs 1.0×, 3.0× and 8.4× the per-size base at 16, 24 and 28 bits.
  - The §5a band orders the two at every size from 18 to 28 bits, at all three weights.
  - At 17 bits the fixed base is cheaper; at 16 bits they are not separated.
- **Distance remaining.** Rho is cheaper at every size and band weight: the per-size pair table costs 6.2× rho at 16 bits, 32× at 32 bits and 78× at 40 bits. No curve has an amortization window (0 of 25).
- **Trend.** The pair table's exponent rises with size: 0.558, then 0.595, then 0.646, with separated intervals. Index calculus moves away from rho as the curves grow.

Scope and limits

- **Curves and claims.** Public synthetic curves and targets; no security claim.
- **Counts only.** No wall time is used. Captured bitop/v1 model counts do not establish complete instrumentation or physical implementation cost.
- **Curve sample.** One curve per size.
- **Deviations.** Four are recorded in `PROTOCOL.md`: the cards were split by the agent harness's time limit, the extension files were renamed, the fits are per tier, and the A/A control is degenerate.
- **Provenance.** The recorded git commit varies between sessions, because documentation commits landed during runs. The binary is identical throughout.